C++

Scenario-Based Coding Problems

All 10 Scenarios — Java → C++ Translation

Priority Queue · Linked List · Stack · Queue · HashMap · Simulation

Hospital · Elevator · Printer · Browser · Editor · Bank · Call Center · Library · Parking · Inventory

iMocha & Interview Coding Preparation · 2025

TABLE OF CONTENTS

Java vs C++ — Key Differences — STL containers, comparators, auto keyword

Scenario 1 — Hospital Emergency Room — Priority Queue / Max-Heap

Scenario 2 — Smart Elevator System — Priority Queue / SCAN Algorithm

Scenario 3 — Print Job Scheduler — Priority Queue + Custom Comparator

Scenario 4 — Browser History — Stack (Back / Forward)

Scenario 5 — Undo/Redo Text Editor — Stack

Scenario 6 — Bank Teller Queue — Min-Heap Load Balancing

Scenario 7 — Call Center Agent Routing — Queue + Multiple Levels

Scenario 8 — Library Book Management — unordered_map + Nested Data

Scenario 9 — Parking Lot System — map + 2D Lookup

Scenario 10 — Inventory Restock Alert — unordered_map + Priority Queue

Quick Reference — Java → C++ STL Cheat Sheet

0 Java vs C++ — Key Differences

Before you read the scenarios

Concept	Java	C++ (STL)
Max-Heap (Priority Queue)	new PriorityQueue<>((a,b)->b.val-a.val)	priority_queue<T> (default max-heap)
Min-Heap	new PriorityQueue<>((a,b)->a.val-b.val)	priority_queue<T,vector<T>,greater<T>>
Custom comparator	Lambda: (a,b)->...	Struct with bool operator()(a,b)
HashMap	HashMap<K,V>	unordered_map<K,V>
Sorted Map	TreeMap<K,V>	map<K,V>
HashSet	HashSet<T>	unordered_set<T>
Queue (FIFO)	new LinkedList<>()	queue<T>
Stack (LIFO)	new Stack<>()	stack<T>
Dynamic array	ArrayList<T>	vector<T>
String	String (object)	string (value type)
Null check	obj == null	ptr == nullptr (or check empty)
Auto type	var (Java 10+)	auto
Print	System.out.println()	cout << ... << endl
Array size	.size() / .length	.size() for STL containers

C++ Priority Queue Comparator Pattern

```
// Custom comparator — this is the most important pattern struct Compare { bool operator()(const Patient& a, const Patient& b) { // Return TRUE if b should come BEFORE a (reverse of intuition) if (a.severity != b.severity) return a.severity < b.severity; // Max-heap by severity return a.arrivalTime > b.arrivalTime; // Tiebreak: earlier first } }; priority_queue, Compare> pq;
```

✓ ***Key insight: In C++ comparator, return true means 'b has higher priority than a'. This is the OPPOSITE of Java's comparator which returns negative for higher priority. Think of it as: return true = swap these two elements.***

1 Hospital Emergency Room

Priority Queue / Max-Heap

A hospital emergency room receives patients throughout the day. Each patient is assigned a severity score: 1 = minor injury, 5 = life-threatening. The doctor always treats the most severe patient first. Tiebreaker: earlier arrival treated first.

Data Structure: Max-Heap with custom comparator

```
#include #include #include using namespace std; struct Patient { string name; int severity; // 1-5, higher = treat first int arrivalTime; // lower = arrived earlier }; // Custom comparator for max-heap by severity, tiebreak by arrival struct PatientCompare { bool operator()(const Patient& a, const Patient& b) { if (a.severity != b.severity) return a.severity < b.severity; // higher severity = higher priority return a.arrivalTime > b.arrivalTime; // earlier arrival = higher priority } }; class EmergencyRoom { priority_queue, PatientCompare> pq; int timer = 0; public: void addPatient(string name, int severity) { pq.push({name, severity, timer++}); cout << name << " added with severity " << severity << endl; } void treatNext() { if (pq.empty()) { cout << "No patients waiting" << endl; return; } Patient p = pq.top(); pq.pop(); cout << "Treating: " << p.name << " (severity=" << p.severity << ")" << endl; } bool hasPatients() { return !pq.empty(); } }; int main() { EmergencyRoom er; er.addPatient("Ali", 2); er.addPatient("Sara", 5); er.addPatient("Ahmed", 5); er.addPatient("Bilal", 1); er.treatNext(); // Sara (severity 5, arrived before Ahmed) er.treatNext(); // Ahmed (severity 5) er.treatNext(); // Ali (severity 2) er.treatNext(); // Bilal (severity 1) return 0; }
```

✓ ***Key insight: C++ priority_queue is a MAX-heap by default. Custom struct comparator operator() returns true = swap (give b higher priority). Struct initializer {name, severity, timer++} works because Patient is an aggregate struct.***

2 Smart Elevator System

Two Priority Queues / SCAN Algorithm

Elevator on floor 1 going UP. Process upward floors first (lowest first = min-heap). When no up requests remain, reverse direction and process down requests (highest first = max-heap).

Data Structure: Two priority_queues (min-heap + max-heap)

```
#include #include #include using namespace std; class Elevator { int currentFloor = 1; bool goingUp = true; // Going up: min-heap (process lowest floor first) priority_queue, greater> upQueue; // Going down: max-heap (process highest floor first) priority_queue downQueue; public: void addRequest(int floor) { if (floor >= currentFloor) upQueue.push(floor); else downQueue.push(floor); cout << "Request added for floor " << floor << endl; } void processAll() { while (!upQueue.empty() || !downQueue.empty()) { if (goingUp) { while (!upQueue.empty()) { currentFloor = upQueue.top(); upQueue.pop(); cout << "Going UP -> Floor " << currentFloor << endl; } goingUp = false; // reverse } else { while (!downQueue.empty()) { currentFloor = downQueue.top(); downQueue.pop(); cout << "Going DOWN -> Floor " << currentFloor << endl; } goingUp = true; } } }; int main() { Elevator e; e.addRequest(3); e.addRequest(7); e.addRequest(2); e.addRequest(8); e.processAll(); // Floor 2 -> 3 -> 7 -> 8 (all going up) return 0; }
```

✓ ***Key insight: priority_queue<int, vector<int>, greater<int>> is a MIN-heap in C++. Default priority_queue<int> is MAX-heap. Two heaps model the SCAN (elevator) algorithm.***

3 Print Job Scheduler

Priority Queue + Two-Level Comparator

High priority jobs print first. Tiebreaker: fewer pages print first to maximize throughput. Jobs: Report(50pg, p1), Invoice(5pg, p3), Email(2pg, p2), Contract(30pg, p3).

Data Structure: Max-heap by priority, min by pages on tie

```
#include #include #include using namespace std; struct PrintJob { string name; int pages; int priority; // 1=low, 3=high }; struct JobCompare { bool operator()(const PrintJob& a, const PrintJob& b) { if (a.priority != b.priority) return a.priority < b.priority; // higher priority first return a.pages > b.pages; // fewer pages first (tiebreak) } }; class PrintScheduler { priority_queue, JobCompare> pq; public: void submit(string name, int pages, int priority) { pq.push({name, pages, priority}); } void printNext() { if (pq.empty()) { cout << "Queue empty" << endl; return; } PrintJob job = pq.top(); pq.pop(); cout << "Printing: " << job.name << " | Pages: " << job.pages << " | Priority: " << job.priority << endl; } int pending() { return pq.size(); } }; int main() { PrintScheduler ps; ps.submit("Report", 50, 1); ps.submit("Invoice", 5, 3); ps.submit("Email", 2, 2); ps.submit("Contract", 30, 3); while (ps.pending() > 0) ps.printNext(); // Invoice -> Contract -> Email -> Report return 0; }
```

✓ ***Key insight: Two-level comparator chained in one operator(). Return true = give b higher priority. Same struct initializer syntax {name, pages, priority} works for aggregate structs.***

4 Browser History — Back / Forward

Two Stacks (LIFO)

Visit pages, go back, go forward. New visit after going back clears forward history. Simulate: google → youtube → github → back → back → stackoverflow → back.

Data Structure: stack<string> for back and forward

```
#include #include #include using namespace std; class BrowserHistory { stack backStack; stack forwardStack; string current; public: BrowserHistory(string homepage) : current(homepage) {} void visit(string url) { backStack.push(current); while (!forwardStack.empty()) forwardStack.pop(); // clear forward current = url; cout << "Visited: " << url << endl; } string back() { if (backStack.empty()) return current; forwardStack.push(current); current = backStack.top(); backStack.pop(); cout << "Back to: " << current << endl; return current; } string forward() { if (forwardStack.empty()) return current; backStack.push(current); current = forwardStack.top(); forwardStack.pop(); cout << "Forward to: " << current << endl; return current; } string getCurrent() { return current; } }; int main() { BrowserHistory b("home"); b.visit("google"); b.visit("youtube"); b.visit("github"); b.back(); // youtube b.back(); // google b.visit("stackoverflow"); // clears forward (youtube, github gone) b.back(); // google (not youtube) return 0; }
```

✓ ***Key insight: C++ stack uses top()+pop() instead of Java's pop(). No single-call 'poll' — must call top() to read, then pop() to remove. Clear a stack by looping while !empty().***

5 Undo / Redo Text Editor

Two Stacks (LIFO)

Type H-e-l-l-o, UNDO twice (removes o,l), type p and !, REDO (does nothing). Final output: 'Help!'

Data Structure: stack<char> for text and undo buffer

```
#include #include #include using namespace std; class TextEditor { stack text; stack undoStack; public: void type(char c) { text.push(c); while (!undoStack.empty()) undoStack.pop(); // clear redo } void undo() { if (!text.empty()) { undoStack.push(text.top()); text.pop(); } } void redo() { if (!undoStack.empty()) { text.push(undoStack.top()); undoStack.pop(); } } string getText() { // Stack doesn't support iteration — copy to get text in order stack temp = text; string result = ""; string reversed = ""; while (!temp.empty()) { reversed += temp.top(); temp.pop(); } // Reverse since stack is LIFO for (int i = reversed.size()-1; i >= 0; i--) result += reversed[i]; return result; } }; int main() { TextEditor e; e.type('H'); e.type('e'); e.type('l'); e.type('l'); e.type('o'); e.undo(); e.undo(); // removes 'o' then 'l' e.type('p'); e.type('!'); // clears redo stack e.redo(); // does nothing cout << e.getText() << endl; // Help! return 0; }
```

✓ ***Key insight: C++ stack has no iterator — to read all elements, copy the stack, then pop and build string in reverse. Same two-stack logic as browser history, just with chars instead of strings.***

6 Bank Teller Queue

Min-Heap Load Balancing

3 tellers. Each new customer goes to the teller with the least total workload. Use a min-heap of (load, tellerID) pairs.

Data Structure: priority_queue<pair<int,int>> min-heap

```
#include #include #include using namespace std; class BankSimulator { // Min-heap: pair priority_queue, vector>, greater>> tellers; public: BankSimulator(int numTellers) { for (int i = 1; i <= numTellers; i++) tellers.push({0, i}); // {load=0, id=i} } void assignCustomer(int custId, int serviceTime) { auto [load, id] = tellers.top(); tellers.pop(); // C++17 structured binding cout << "Customer " << custId << " -> Teller " << id << " (load: " << load << " min)" << endl; tellers.push({load + serviceTime, id}); } void printFinalLoads() { cout << "\nFinal workload:" << endl; // Copy to avoid modifying original auto temp = tellers; vector> sorted; while (!temp.empty()) { sorted.push_back(temp.top()); temp.pop(); } sort(sorted.begin(), sorted.end(), [](auto& a, auto& b){ return a.second < b.second; }); for (auto& [load, id] : sorted) cout << "Teller " << id << ": " << load << " min" << endl; } }; int main() { BankSimulator bank(3); int times[] = {3, 5, 2, 4, 1, 6}; for (int i = 0; i < 6; i++) bank.assignCustomer(i+1, times[i]); bank.printFinalLoads(); return 0; }
```

✓ ***Key insight: pair<int,int> compares first by first element then second — so pair<load, id> min-heap automatically sorts by load first. auto [load, id] = tellers.top() is C++17 structured binding (cleaner than .first/.second).***

7 Call Center Agent Routing

queue<string> per Level

3 support levels, 2 agents each. Incoming call goes to first free Level 1 agent. If Level 1 full, try Level 2, then Level 3. If all busy — waiting.

Data Structure: array of queue<string> (one per level)

```
#include #include #include using namespace std; class CallCenter { queue levels[3]; // levels[0]=L1, levels[1]=L2, levels[2]=L3 public: CallCenter() { levels[0].push("L1-Agent1"); levels[0].push("L1-Agent2"); levels[1].push("L2-Agent1"); levels[1].push("L2-Agent2"); levels[2].push("L3-Agent1"); levels[2].push("L3-Agent2"); } string dispatch(int callId) { for (int lvl = 0; lvl < 3; lvl++) { if (!levels[lvl].empty()) { string agent = levels[lvl].front(); levels[lvl].pop(); cout << "Call " << callId << " -> " << agent << " (Level " << (lvl+1) << ")" << endl; return agent; } } cout << "Call " << callId << " -> All agents busy, waiting" << endl; return ""; } void freeAgent(string agent, int level) { levels[level].push(agent); } }; int main() { CallCenter cc; cc.dispatch(101); // L1-Agent1 cc.dispatch(102); // L1-Agent2 cc.dispatch(103); // L2-Agent1 (L1 full) cc.dispatch(104); // L2-Agent2 cc.dispatch(105); // L3-Agent1 cc.dispatch(106); // L3-Agent2 cc.dispatch(107); // All busy, waiting return 0; }
```

✓ ***Key insight: C++ queue uses front()+pop() (not top()). Array of queues queue<string> levels[3] is cleaner than vector<queue>. Loop through levels in order — first non-empty level serves the call.***

8 Library Book Management

unordered_map + Nested Data

Manage books by ISBN. Track which member has which books. Borrow fails if book unavailable. Return restores availability.

Data Structure: unordered_map<string, Book> + map<string, vector<string>>

```
#include #include #include #include #include using namespace std; struct Book { string isbn, title; bool available = true; string borrowedBy = ""; // empty if available }; class Library { unordered_map catalog; unordered_map> memberBooks; public: void addBook(string isbn, string title) { catalog[isbn] = {isbn, title, true, ""}; } bool borrow(string memberId, string isbn) { if (catalog.find(isbn) == catalog.end()) { cout << "Book not found" << endl; return false; } Book& book = catalog[isbn]; if (!book.available) { cout << book.title << " not available, borrowed by: " << book.borrowedBy << endl; return false; } book.available = false; book.borrowedBy = memberId; memberBooks[memberId].push_back(isbn); // creates vector if not exists cout << memberId << " borrowed: " << book.title << endl; return true; } bool returnBook(string memberId, string isbn) { if (catalog.find(isbn)==catalog.end() || catalog[isbn].available) { cout << "Invalid return" << endl; return false; } Book& book = catalog[isbn]; book.available = true; book.borrowedBy = ""; auto& books = memberBooks[memberId]; books.erase(remove(books.begin(), books.end(), isbn), books.end()); cout << memberId << " returned: " << book.title << endl; return true; } void showMemberBooks(string memberId) { auto& books = memberBooks[memberId]; cout << memberId << " has: " << books.size() << " book(s)" << endl; for (auto& isbn : books) cout << " - " << catalog[isbn].title << endl; } }; int main() { Library lib; lib.addBook("978-1", "Clean Code"); lib.addBook("978-2", "Design Patterns"); lib.borrow("M001", "978-1"); lib.borrow("M002", "978-1"); // fails - unavailable lib.borrow("M002", "978-2"); lib.showMemberBooks("M001"); lib.returnBook("M001", "978-1"); lib.borrow("M002", "978-1"); // now works return 0; }
```

✓ ***Key insight: catalog[isbn] auto-creates entry if missing (use find() to check first). Book& = reference — changes affect the original map entry. erase+remove idiom removes a value from vector (Java's .remove() equivalent).***

9 Parking Lot System

map + 2D Nested Lookup

3 levels, 10 spots each. Small = any spot. Large = spots 6-10. Motorcycle = 2 per spot. Park nearest available spot. Remove by plate.

Data Structure: map<int, map<int,string>> + map<string,int> for moto

```
#include #include #include using namespace std; class ParkingLot { map> lot; // level -> spot -> plate map motoCount; // "L-S" -> count (max 2) int levels, spotsPerLevel; public: ParkingLot(int lvls, int spots) : levels(lvls), spotsPerLevel(spots) {} bool park(string plate, string type) { for (int level = 1; level <= levels; level++) { int start = (type == "LARGE") ? 6 : 1; for (int spot = start; spot <= spotsPerLevel; spot++) { string key = to_string(level) + "-" + to_string(spot); if (type == "MOTO") { int count = motoCount.count(key) ? motoCount[key] : 0; if (count < 2) { motoCount[key] = count + 1; string existing = lot[level].count(spot) ? lot[level][spot] : ""; lot[level][spot] = existing.empty() ? plate : existing+","+plate; cout << plate << " parked at L" << level << "-S" << spot << endl; return true; } } else if (lot[level].find(spot) == lot[level].end()) { lot[level][spot] = plate; cout << plate << " parked at Level " << level << " Spot " << spot << endl; return true; } } } cout << "No spot available for " << type << endl; return false; } void leave(string plate) { for (auto& [level, floor] : lot) { for (auto& [spot, plates] : floor) { if (plates.find(plate) != string::npos) { cout << plate << " left from L" << level << "-S" << spot << endl; floor.erase(spot); return; } } } }; int main() { ParkingLot pl(3, 10); pl.park("ABC-001", "SMALL"); pl.park("XYZ-002", "LARGE"); pl.park("MOT-001", "MOTO"); pl.park("MOT-002", "MOTO"); // shares spot with MOT-001 pl.leave("ABC-001"); return 0; }
```

✓ ***Key insight: Nested map<int,map<int,string>> gives O(log n) lookup per level. map.count(key) checks existence without inserting (safer than [] which creates entry). C++17 structured bindings: for(auto& [level, floor] : lot) is cleaner than iterator syntax.***

10 Inventory Restock Alert

unordered_map + Min-Heap

Track stock levels. Auto-trigger restock alert when stock drops below threshold. Min-heap prioritizes most urgent items (lowest stock). Avoid duplicate alerts.

Data Structure: unordered_map<string,Product*> + priority_queue min-heap

```
#include #include #include #include #include #include using namespace std; struct Product { string id; int stock; int threshold; }; struct ProductCompare { bool operator()(const Product* a, const Product* b) { return a->stock > b->stock; // min-heap: lowest stock = highest priority } }; class Inventory { unordered_map products; priority_queue, ProductCompare> restockQueue; unordered_set inQueue; // prevent duplicates public: void addProduct(string id, int stock, int threshold) { products[id] = new Product{id, stock, threshold}; } void sell(string id, int qty) { if (products.find(id) == products.end()) { cout << "Product not found" << endl; return; } Product* p = products[id]; p->stock = max(0, p->stock - qty); cout << id << " stock: " << p->stock << endl; if (p->stock < p->threshold && inQueue.find(id) == inQueue.end()) { restockQueue.push(p); inQueue.insert(id); cout << " RESTOCK ALERT: " << id << endl; } } void processRestock(int qty) { if (restockQueue.empty()) { cout << "Nothing to restock" << endl; return; } Product* p = restockQueue.top(); restockQueue.pop(); inQueue.erase(p->id); p->stock += qty; cout << "Restocked " << p->id << " -> stock now: " << p->stock << endl; } void showRestockList() { cout << "Needs restock: "; for (auto& id : inQueue) cout << id << " "; cout << endl; } ~Inventory() { for (auto& [id, p] : products) delete p; } }; int main() { Inventory inv; inv.addProduct("ITEM-A", 50, 20); inv.addProduct("ITEM-B", 30, 25); inv.addProduct("ITEM-C", 100, 10); inv.sell("ITEM-A", 40); // stock=10, alert inv.sell("ITEM-B", 20); // stock=10, alert inv.sell("ITEM-C", 95); // stock=5, alert inv.showRestockList(); inv.processRestock(100); // ITEM-C (stock=5, most urgent) inv.processRestock(100); // ITEM-A or ITEM-B (both at 10) return 0; }
```

✓ ***Key insight: Use pointers (Product*) in the priority_queue so that when stock changes via sell(), the heap sees updated values. unordered_set for O(1) duplicate check. Destructor ~Inventory() cleans up heap memory — good practice in C++.***

REF

Java → C++ STL Quick Reference

Cheat Sheet

Container Operations

Operation	Java	C++
Add to end	.add(x) / .offer(x)	.push(x) or .push_back(x)
Remove from front	.poll()	.front() then .pop()
Remove from top	.poll() [PQ]	.top() then .pop()
Peek front	.peek()	.front() [queue] / .top() [stack/PQ]
Check empty	.isEmpty()	.empty()
Size	.size()	.size()
Find in map	.containsKey(k)	.count(k) or .find(k) != .end()
Get from map	.get(k)	[k] (creates if missing!) or .at(k)
Remove from map	.remove(k)	.erase(k)
Remove from vector	list.remove(obj)	erase(remove(v.begin(),v.end(),x),v.end())
Iterate map	for(Map.Entry e : map.entrySet())	for(auto& [k,v] : map)
Sort vector	Collections.sort(list)	sort(v.begin(), v.end())
Lambda sort	(a,b)->a.val-b.val	[](auto& a, auto& b){ return a.val<b.val; }

Priority Queue Patterns in C++

Type	Declaration
Max-heap (int)	priority_queue<int> pq;
Min-heap (int)	priority_queue<int, vector<int>, greater<int>> pq;
Max-heap (pair)	priority_queue<pair<int,int>> pq;
Min-heap (pair)	priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
Custom struct max	priority_queue<MyType, vector<MyType>, MyCompare> pq;
Push	pq.push(element);
Access top	pq.top();
Remove top	pq.pop();

Keyword → Data Structure (C++)

Keyword in Problem	Use This in C++

'process in order', 'first come first served'	queue<T>
'highest priority first', 'most severe'	priority_queue<T, vector<T>, Compare> (max-heap)
'lowest first', 'minimum', 'least loaded'	priority_queue<T, vector<T>, greater<T>> (min-heap)
'undo', 'back button', 'last action'	stack<T>
'look up by key', 'fast search', 'O(1) lookup'	unordered_map<K,V>
'sorted order', 'alphabetical'	map<K,V> (sorted by key)
'unique values', 'no duplicates'	unordered_set<T>
'nearest available', 'least loaded teller'	priority_queue min-heap with pairs
'insert/delete in middle', 'chain'	list<T> (doubly linked list)

✓ ***Final note: The logic in every scenario is IDENTICAL between Java and C++. Only the syntax differs. Master the pattern in one language — translating to the other is just replacing PriorityQueue with priority_queue, HashMap with unordered_map, and poll() with top()+pop().***